End-to-End Inventory Control via Monotonic Kolmogorov–Arnold Networks

Qi Zhang
Technical University of Munich
Munich, Germany

Keming Lin
Technical University of Munich
Munich, Germany

***Abstract*—Neural inventory policies can achieve high average profit while violating expected response directions. We study monotonicity constraints in the lost-sales setting of Maggiar et al., comparing MonoKAN with an unconstrained KAN under matched training protocols within each policy design. In the main design, the network predicts order quantity from inventory, demand history and economics. The unconstrained KAN orders more as inventory rises on 8.7%–51.3% of products across six runs; MonoKAN eliminates these reversals. The profit effect depends on the initialization scheme: with constant initial outputs, MonoKAN earns 14.11 less per period (95% CI for MonoKAN minus KAN: $[-22.32, -5.90]$), whereas with open output layers it earns 548 more because the unconstrained KAN trains poorly. These constraints guarantee response directions, not appropriate response magnitudes. In an alternative design, a network chooses a demand percentile and a differentiable decoder produces an order-up-to target. Inventory monotonicity then follows from the decoder, while MonoKAN constrains the economic mapping. The final alternative policy reaches 0.80% mean per-product regret; its mean-profit difference from KAN is not statistically significant. Classical rules with the same demand information remain stronger on regret. The results show that the operational effect of monotonic constraints depends on initialization and policy parameterization; they do not isolate a causal relationship between the number of constrained inputs and profit.**

***Index Terms*—Certified monotonicity, end-to-end learning, inventory control, Kolmogorov–Arnold networks, lost sales, newsvendor problem**

I. Introduction

Every day a retailer decides how many units of each product to order. Too many, and the surplus ties up money and shelf space; too few, and customers leave empty-handed. The classical approach estimates the demand distribution first and then derives the order from it. End-to-end learning skips the separate forecast: a neural network observes the state of the system, including the stock on hand, recent demand and the product's prices and costs, and outputs the order directly, trained to maximise simulated profit. Maggiar et al. [1] show that such a policy, trained on synthetic products with known optimal benchmarks, comes within 0.41% of the omniscient benchmark in average profit.

Profit optimization alone does not ensure that the learned policy follows expected comparative statics. For example, a policy may order more when inventory rises, holding demand history and economics fixed. Monotonic architectures can enforce such directions for designated inputs [2], [3]. MonoKAN [4] is such an architecture: a Kolmogorov–Arnold network [5] whose learned edge functions are constrained to be monotone, which certifies the direction of every designated input for all possible inputs.

We ask: *how do monotonic constraints affect the behaviour and profit of an end-to-end inventory policy?* We compare MonoKAN with an unconstrained KAN of the same architecture within each design. For direct quantity, both models are tested under constant and open initialization schemes with the same calibration target (Section IV-A); projection can still change their initial functions.

Following the examiner's feedback, the main design predicts the order quantity directly, so inventory monotonicity must be enforced by the network. The alternative predicts a demand percentile and orders up to a learned target; inventory monotonicity then follows from the decoder. This separates two uses of inventory knowledge, although the comparison between designs changes several factors and is not an isolated ablation of the inventory constraint.

The project contributes:

- a direct-quantity MonoKAN policy that certifies the direction of all 37 inputs, and a matched comparison with the same KAN without the certificate and with the feed-forward network of Maggiar et al.;
- measurements of inventory reversals and profit differences under matched initialization schemes;
- a soft-quantile target-stock policy with a monotone level network, the steps that took it from 21.4% to 0.80% regret, and comparisons with classical rules that use the same information;
- supporting analyses of economic responses, model size, decision time and the tested training-set sizes.

II. Setting

A. The Lost-Sales Inventory Problem

We use the synthetic lost-sales setting of Maggiar et al. [1]. Each product has a selling price p, a purchase cost c, a holding cost h per unit left over at the end of a period and a penalty k per unit of unmet demand. Orders arrive immediately (no lead time) and unmet demand is lost. In period t the policy

Final report of an Interdisciplinary Project at the Chair of Logistics and Supply Chain Management, TUM School of Management, Technical University of Munich, examined by Prof. Dr. Stefan Minner. Munich, October 2026.

sees the stock on hand I_t and orders $q_t \geq 0$; then demand d_t is realised, and the period's profit and the next stock are

$$\begin{aligned} r_t &= p \min(d_t,\, I_t + q_t) - c\, q_t \\ &\quad - h\,(I_t + q_t - d_t)^+ - k\,(d_t - I_t - q_t)^+, \\ I_{t+1} &= (I_t + q_t - d_t)^+. \end{aligned} \tag{1}$$

Demand is independent and identically distributed over time. Gamma samples, with product-specific mean μ and coefficient of variation ν, are rounded to nonnegative integers in the implementation. These parameters are hidden: a learned policy observes economics and realized demand, initially 32 past observations. Demand is observed even when sales are lost; this is not a censored-sales observation model. Products are drawn as in the paper: p exponential with mean 100, $c = p \cdot U$, h exponential with mean 5, $k = 10 \cdot U$, μ exponential with mean 100 and ν uniform on $(0, 1)$, with independent uniform draws U.

For the corresponding continuous-demand model, a base-stock policy orders up to the critical-fractile quantile of the demand distribution [6],

$$\begin{aligned} S^* &= F^{-1}(\beta), \qquad \beta = \frac{p - c + k}{p - c + k + h}, \\ q_t &= (S^* - I_t)^+. \end{aligned} \tag{2}$$

This is the newsvendor solution. With the true Gamma parameters it defines the *omniscient* benchmark: a distribution-informed base-stock policy that knows the demand distribution but not the future demand. It is our reference point, not a strict pathwise upper bound. It retains the continuous Gamma quantile rather than solving a separate rounded-demand optimum; another policy can do better on a particular finite demand path. Classical non-omniscient rules estimate $F^{-1}(\beta)$ from observed demand, either as an empirical quantile or by fitting a Gamma distribution (Section IV-C).

B. Training End to End Through the Simulator

All learned policies are trained by DirectBackprop, as in the paper: the simulator in (1) is written in a differentiable form, a policy runs a batch of products for a 100-period training horizon, and the gradient of the profit objective flows through the simulated dynamics into the network's parameters. Each training product has a fixed demand path; policies use only observations preceding the current action. Terminal stock receives a purchase-cost credit, avoiding an artificial end-of-horizon write-off; holding costs still apply. Adam uses learning rate 0.001. Each logged epoch is one optimizer update on a batch of 512 products sampled with replacement, rather than a full pass through the catalogue. Runs use three training seeds (2025, 2026, 2027), with product-generation seed fixed. Checkpoints maximize the validation version of the training objective: unweighted profit in the main design and scaled profit in the final alternative.

C. Measuring Performance

Every policy is evaluated on the same 300 validation products and the same demand paths, over 520 periods of which the first 20 are discarded to reduce initial-state effects. We report two measures.

Profit per period, averaged over products, is the paper's measure. Product economics are heavy tailed (profit per period has mean 4,284 and standard deviation 8,537 over products), so this average is dominated by a few large products.

Regret is the mean per-product shortfall against the omniscient policy, in percent:

$$\text{regret} = \frac{100}{|\mathcal{P}^+|} \sum_{i \in \mathcal{P}^+} \frac{R_i^* - R_i}{R_i^*}, \tag{3}$$

where R_i and R_i^* are the profits of the policy and the omniscient policy on product i. Every product counts the same, so regret reveals failures on typical products that the average hides. The set $\mathcal{P}^+$ excludes the 21 of 300 products on which even the omniscient policy makes no profit, where a relative shortfall is meaningless. The two measures can disagree: the initial direct-target model trailed a classical rule by 3.6% in mean profit, while its mean per-product relative shortfall was 21.4%. This is an average relative loss, not a median-product result.

Because all policies face identical products and demand, we compare them *paired by product*, which removes the large product-to-product spread. For a difference in profit we first average each product's profit over the three training seeds and then compare product by product. We report the mean difference with its 95% confidence interval and the paired t-test, the Wilcoxon signed-rank test, which is less sensitive to a few very large products, and the share of products on which one policy is ahead. These tests treat the trained models as given: they describe variation across products, not the variation from retraining with new seeds, which three seeds can only indicate.

III. Certified Monotonicity With MonoKAN

A. Kolmogorov–Arnold Networks

A multilayer perceptron applies fixed nonlinearities at its nodes and learns linear weights on its edges. A Kolmogorov–Arnold network (KAN) does the reverse: every edge carries its own learnable univariate function, and nodes only sum their incoming edges [5]. A layer maps $x \in \mathbb{R}^n$ to

$$y_j = \sum_i \varphi_{ji}(x_i), \qquad \varphi_{ji}(x) = w_{ji}^b\, b(x) + w_{ji}^s\, s_{ji}(x), \tag{4}$$

where b is a fixed increasing base function (a shifted, positive ELU in our implementation) and s_{ji} is a learned cubic Hermite spline on a grid of knots, with the value and slope at each knot as parameters. Because each edge is a curve of one variable, a trained KAN can be inspected edge by edge.

B. Monotone Splines and the Certificate

A sufficient condition for a cubic Hermite interval to be monotone is the Fritsch–Carlson condition [7]: for nonzero secant slope Δ, the normalized endpoint slopes satisfy $\alpha, \beta \geq 0$ and $\alpha^2 + \beta^2 \leq 9$. Flat intervals have zero endpoint slopes. MonoKAN [4] uses this to certify monotonicity. After

every optimiser step we project the parameters back onto the monotone set:

- for every input with a required direction, the first-layer edges get a base weight of that sign, a non-negative spline weight, and a spline projected to be monotone in that direction (knot values made monotone, knot slopes given the right sign and scaled until the Fritsch–Carlson condition holds);
- every edge in later layers is projected to be increasing, with non-negative weights.

A sum of functions that move in one direction moves in that direction, and an increasing function of it keeps it. The network's output is therefore monotone in every designated input, with the designated sign, for every possible input, not only on the training data, up to floating-point rounding. Outside the knot range, linear extrapolation uses the signed endpoint slopes. The guarantee concerns direction, not derivative magnitude or profit.

C. Two Implementation Lessons

Two defects in our first implementation are worth recording, because both changed results. First, the projection wrote the conditions of neighbouring intervals one after the other, although adjacent intervals share a knot slope. The second write could undo the first, leaving rare, tiny violations (worst case $5.6 \cdot 10^{-4}$). The corrected projection combines both conditions at each knot, and a regression test checks it on uneven grids. Second, every spline originally lived on a fixed grid from -2 to 4, while the normalised inputs did not. 63.8% of demand observations fell into the first of seven intervals, and holding cost and lost-sale penalty spanned less than one interval. Those splines could barely bend, and the first model learned an almost constant policy. Placing the knots at quantiles of the training data, the spline analogue of the grid update of Liu et al. [5], more than halved the regret of our initial direct-target model (Section VI-A). After grid calibration, projection is reapplied using the actual knot spacings to restore the monotonicity conditions.

IV. Two Policy Designs

Fig. 1 shows the two policy designs that we compare.

A. Main Design: The Network Decides the Order

The main design maps stock on hand, the last 32 demands and four economic quantities to the order: $1 + 32 + 4 = 37$ inputs. With immediate arrivals there is no pipeline inventory. The required direction for each input is fixed in advance (Table I), and MonoKAN certifies all 37. The demand inputs are sorted before entering the network. Each sorted value is nondecreasing in every raw observation, so the certified directions carry over to the raw inputs. The output passes through a softplus and is multiplied by the mean of the 32 observations available at the start, which makes the order non-negative and proportional to the product's scale without changing any direction. In evaluation, but not in training, the order is limited to $\max(4m_0 - I_t, 0)$, where m_0 is the initial historical mean floored at 1. If stock already exceeds this threshold, the guard prevents further orders but cannot remove existing stock. The guard preserves the specified directions and applies to all arms; Section V-B reports its effect.

TABLE I
Inputs of the Main Design and Their Certified Directions

Input	Count	Certified rule	Reason
Stock on hand	1	more stock → never order more	stock already covers demand
Demand in each of the last 32 periods	32	more demand → never order less	higher demand needs more stock
Selling price	1	higher → never order less	a lost sale forgoes more margin
Lost-sale penalty	1	higher → never order less	running out costs more
Purchase cost	1	higher → never order more	each unit earns less margin
Holding cost	1	higher → never order more	leftover stock costs more

MonoKAN uses one hidden layer of 8 nodes and Hermite splines with 5 intervals placed at data quantiles, 4,265 parameters in total. The comparison KAN has the same architecture and training but no projection. Either network can start from one of two initialisations of its output layer. In the *constant* start the output-layer weights are zero, so every product starts at the same order per unit of mean demand: the median empirical newsvendor level of the training products. In the *open* start the weights are nonzero, so the initial order already varies with the inputs, and the bias is shifted so that the median initial order on reference contexts equals that level. Our first runs started MonoKAN open and the KAN constant. Because the start turned out to matter, we tested both networks under each initialization scheme (Section V-B). Constant starts give the same initial actions; open starts share the calibration target but need not give identical initial functions after projection. The third arm is a causal-CNN/FCNN implementation following Maggiar et al. with 2,105 parameters and its own initialisation. All arms are trained on the same 1,000 products for 2,400 epochs to maximise total profit.

To check the stock rule we sweep the stock on hand of every validation product from 0 to 4 days of mean demand in 25 steps, holding all other inputs at the product's observed values, and flag every step at which the order rises (by more than 10^{-4} of a day's demand). For each product, a linear fit over stock from 0 to 1.5 days of mean demand measures the average response; coefficients are then averaged over products and seeds. This statistic is not a pointwise derivative bound. An order-up-to policy has derivative -1 while ordering and 0 otherwise; the reference policy's average fitted slope is -0.90.

B. Alternative Design: Order Up to a Learned Target

Equation (2) suggests a different structure: let the network choose only the percentile, and let a fixed formula turn it into an order. A small MonoKAN level network reads only the four economic inputs and outputs a percentile $u = \sigma(f(p, c, h, k))$. The target stock is a smoothed empirical quantile of all

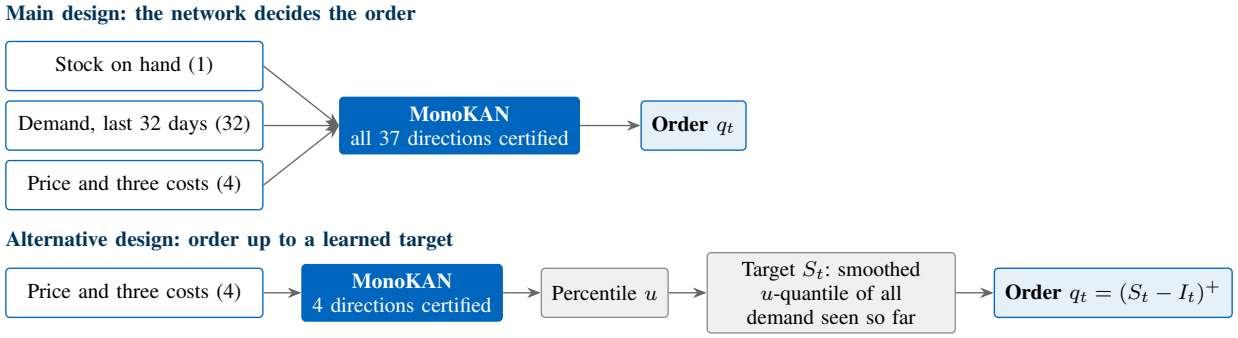


Fig. 1. The two policy designs. In the main design the network decides the order, so the rule "more stock, never a larger order" must come from the network. In the alternative it comes from the subtraction in the last step, and the network only chooses a demand percentile from the economics. Blue: learned; grey: fixed computation.

demand observed so far. With $z_{(1)} \le \cdots \le z_{(n)}$ the sorted observations and $t_j = (j-1)/(n-1)$ their positions,

$$S_t = g(u)\Big[z_{(1)} + \sum_{j=2}^{n} \sigma\Big(\frac{u - t_j}{\tau_n}\Big)\big(z_{(j)} - z_{(j-1)}\big)\Big], \qquad (5)$$
$$q_t = (S_t - I_t)^+,$$

where the final configuration measures smoothing in order-statistic counts: $\tau_n = \max(\mathrm{softplus}(\eta), 10^{-4})/(n-1)$, for $n \ge 2$, with learned η. The positive gain $g(u) = \exp(au + b)$ has learned bias b and nonnegative slope $a = \mathrm{softplus}(\rho)$. The weights $w_j = \sigma((u - t_j)/\tau_n)$ lie in $(0, 1)$ and decrease in j, because the positions t_j increase. Rearranged, the bracket in (5) is

$$(1 - w_2)\, z_{(1)} + \sum_{j=2}^{n-1} (w_j - w_{j+1})\, z_{(j)} + w_n z_{(n)},$$

a combination of the order statistics with non-negative coefficients; each order statistic is nondecreasing in every observed demand, and the positive gain preserves this, so S_t is nondecreasing in every observed demand. S_t increases with u, which MonoKAN makes increase in price and penalty and decrease in purchase and holding cost. And the subtraction makes the order nonincreasing in stock for *any* level network. MonoKAN therefore constrains only the four economic inputs. The level network has two layers of 8 nodes and 7-interval splines, with 1,889 parameters. The complete policy has 1,892, including temperature and the two gain parameters. It is trained on 5,000 products with a scaled objective: each product's simulated profit is divided by $p_i \hat{\mu}_i T$, its price times its observed mean demand times the horizon (each floored at 1), before averaging over products, so that a few large products do not dominate the gradient.

The same decoder can use three kinds of demand information, which matters for fair comparisons: one fixed estimate from the first 32 days, the last 32 days updated every period, or all demand seen so far (the final policy). Maggiar et al. [1, Sec. 5.3.1] suggest such order-up-to structures, but warn that they are fragile: they only work when the optimal policy is known to be of this form.

C. Baselines and the Matched Protocol

Besides the omniscient policy, we use two classical rules that estimate the target from observed demand: the empirical quantile at the critical fractile, and a Gamma distribution fitted by the method of moments. The Gamma fit explicitly assumes a Gamma demand family, whereas the learned policies acquire an implicit prior from Gamma-generated training products. To match demand information, each rule is evaluated with each of the three windows:

- *first 32 days, never updated*: how the paper defines its non-omniscient benchmarks;
- *last 32 days, updated daily*: what a policy in daily use sees, and the demand window of the main design's inputs;
- *all demand seen so far*: the information of the final alternative policy.

Within each MonoKAN–KAN block, the models share products, demand paths, horizon, objective, update budget and architecture. Initialization schemes are matched, except for the explicitly labeled first runs. Across policy designs, other factors differ, including the decoder, architecture, initialization and evaluation guard. Checkpoints are considered every 300 updates for the main design and every 150 for the matched alternative; the cross-design comparison therefore does not isolate a single factor.

V. Results: Main Design

Table II reports the direct-quantity results for both initialization schemes and the reference policies.

A. Without the Certificate, the Stock Rule Breaks

From the constant start, the KAN without the certificate orders more as stock rises on 35.1% of products on average (Fig. 2(a)). This happens in every training run, but how often varies widely: 45.3%, 8.7% and 51.3% of products for the three seeds. The breaks occur at 3.7% of all swept stock steps; most are small, but the largest is an extra order of about 11% of a day's demand. From the open start the KAN breaks the rule on 19.7%, 16.0% and 35.0% of products, but these breaks are tiny, at most 0.2% of a day's demand, because this KAN barely reacts to stock at all (Section V-B). Fig. 2(b) shows

TABLE II
MAIN DESIGN RESULTS

Policy	Start	Profit / period	Regret	Breaks	Reaction	Parameters
MonoKAN (certified)	constant	4,215.20 ± 5.64	14.60%	0.0%	0.46	4,265
Same KAN, no certificate	constant	4,229.31 ± 10.88	13.33%	35.1%	0.54	4,265
MonoKAN (certified)	open	4,206.43 ± 17.56	15.71%	0.0%	0.52	4,265
Same KAN, no certificate	open	3,658.28 ± 147.26	48.78%	23.6%	0.02	4,265
Paper FCNN	—	3,874.38 ± 622.50	20.64%	0.0%	0.34	2,105
Classical rules, last 32 days (empirical quantile / Gamma fit)	—	4,255.93 / 4,275.37	1.90% / 1.25%	0.0%	—	—
Omniscient base stock (knows the true distribution)	—	4,297.45	0.00%	0.0%	0.90	—

Three seeds each, the same 300 validation products. "Start" is the output-layer initialisation (Section IV-A). Profit per period ± standard deviation over seeds. "Breaks" is the share of products on which the order rises with stock at some point of the sweep; "reaction" is the negative of the product- and seed-averaged linear-fit coefficient, not a bound on local derivatives.

the largest reversal we found in any run; the break rates above show how common reversals are. Over part of the stock range, the KAN's order goes *up* as stock goes up. The order of the certified MonoKAN on the same product never rises with stock. MonoKAN breaks the rule for no start, seed or product, as its construction guarantees.

The risk depends on the model family. The paper's FCNN did not break the rule in any of our three runs, while the KAN did in all six runs. MonoKAN excludes this direction reversal by construction. Maggiar et al. [1, Sec. 5.3.3] instead add a soft penalty for violations to the training loss, which makes breaks less likely but cannot exclude them.

B. Profit With and Without the Certificate

Initialization changes the comparison (Table III). With constant starts, MonoKAN earns 14.11 less per period, about 0.3% of mean profit, and regret increases from 13.33% to 14.60%. This comparison concerns all 37 constraints together; it cannot separate the inventory constraint from the other directions, or representational restriction from optimization difficulty. The table reports the product-level confidence intervals and tests.

From the open start the comparison reverses. The unconstrained KAN barely learns to react to stock: its order falls by only 0.02 per unit of stock, against 0.54 from the constant start, its regret is 48.78%, and its profit varies widely between seeds. MonoKAN earns 548.15 more per period (95% CI 390.33 to 705.97) and is ahead on 82% of products. MonoKAN itself depends far less on its start: it earns 8.78 less from the open start than from the constant one (95% CI 0.50 to 17.06), the unconstrained KAN 571 less. We have not investigated why the open start fails for the unconstrained KAN. In these runs the certificate thus did two things: it cost a little profit when both networks started well, and it kept training reliable from a start at which the unconstrained network failed. Our first runs mixed the starts (MonoKAN open, KAN constant) and showed a gap of 22.89; the part beyond the constant-start gap is MonoKAN's own loss from its start.

The evaluation cap of Section IV-A does not drive these results. Evaluated without it, MonoKAN trails the KAN by 15.29 from the constant start (95% CI 6.37 to 24.21) and leads by 749 from the open start. Without the cap every network's regret is higher, by 0.8 to 1.6 points for the three that trained well and far more for the open-start KAN, whose excessive orders the cap cuts.

The FCNN average includes a failed run: seed 2027 earns 3,155.76 (42.43% regret), whereas the other two average 4,233.69. All three remain in the reported result. This shows substantial seed sensitivity in this setup, not a general inferiority of FCNN policies.

C. What the Certificate Cannot Fix

Monotonicity fixes response direction, not magnitude. The reference policy has an average fitted stock reaction of 0.90, compared with at most 0.54 among the direct-quantity networks (Fig. 3). These averages can hide steep local declines and flat tails: the displayed MonoKAN curve, for example, has a local secant slope near -5.20. Thus the networks do not already satisfy a pointwise slope bound of $[-1, 0]$. The results show a mismatch with the order-up-to response shape, but do not isolate its contribution to regret. This distinction between signs and sensitivity magnitudes is also discussed by Maggiar et al. [1, Sec. 5.3.2].

No direct-quantity network comes close to a classical rule that uses the same 32-day window. The empirical quantile and the Gamma fit on the last 32 days reach 1.90% and 1.25% regret, against 13.33% or more for every network. The paper's interpretability test points the same way. A direct-quantity policy's order at zero stock is its implied order-up-to level. Per unit of mean demand, which removes product scale, MonoKAN's implied level correlates only 0.43 to 0.49 with the optimal level, depending on the start, and the unconstrained KAN's 0.62 from the constant start, against 0.93 for the Gamma fit on the same history. In raw units every policy looks good, because product scale alone explains most of the variation. The certificate makes the direction of every input transparent, but the learned levels are only loosely tied to the optimum.

VI. RESULTS: ALTERNATIVE DESIGN

A. From 21.4% to 0.80% Regret

The first MonoKAN in this design, a deep network that output the target stock directly, reached 21.40% regret, worse

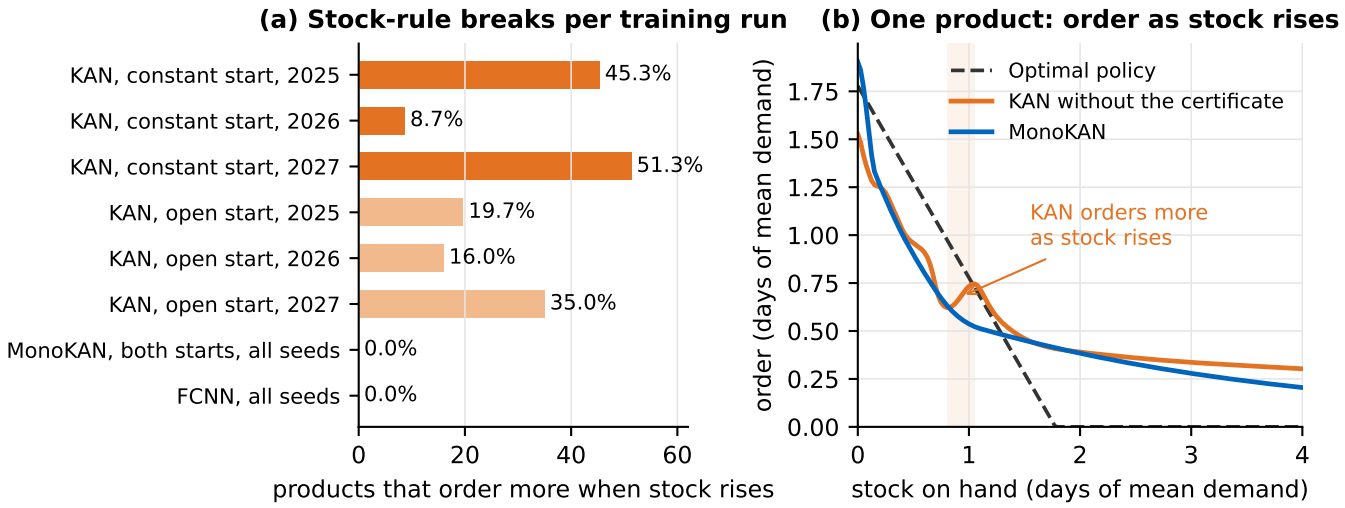


Fig. 2. (a) Share of validation products on which the order rises with stock, for each training run of the KAN without the certificate from the constant and the open start, and for MonoKAN (both starts) and the FCNN. (b) Order against stock on hand for the product and run where the KAN breaks the rule most (constant start, seed 2027), with MonoKAN from the open start and the same seed; both axes in days of mean demand. This worst-reversal illustration is not a matched-initialization comparison. The reference policy orders up to its target one-for-one.

TABLE III
MONOKAN MINUS THE SAME KAN WITHOUT THE CERTIFICATE, MAIN DESIGN

Start of the two networks	Difference	95% CI	t-test p	Wilcoxon p	MonoKAN ahead
Both constant	−14.11	[−22.32, −5.90]	0.0008	< 0.0001	32%
Both open	+548.15	[+390.33, +705.97]	< 0.0001	< 0.0001	82%
MonoKAN open, KAN constant (first runs)	−22.89	[−32.57, −13.20]	< 0.0001	< 0.0001	28%

Profit per period, paired over the 300 products with rewards averaged over seeds.

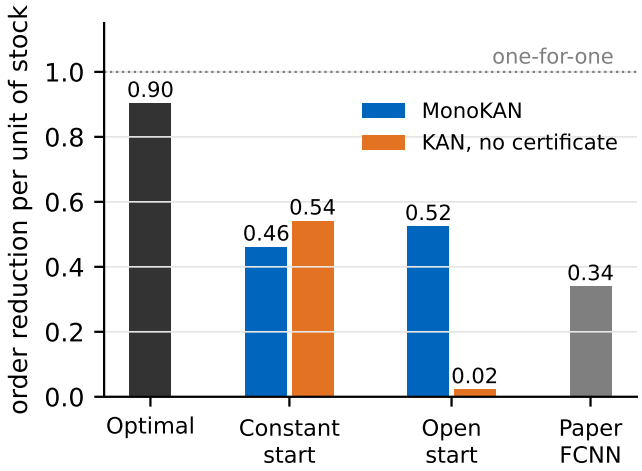


Fig. 3. Negative linear-fit coefficient over stock from 0 to 1.5 days of mean demand, averaged over products and seeds. This summarizes average response strength, not pointwise derivative bounds.

than an unconstrained FCNN in the same structure (9.98%) and about twelve times the regret of a classical rule. An unconstrained KAN of the same architecture was similarly poor (20.68%), so the constraint was not the main cause. Fig. 4 shows cumulative development changes, not independent one-factor ablations. All variants were designed to enforce monotonicity; the projection defect of Section III-C was corrected at the fourth step. The two largest gains came from placing the spline knots where the data lies (Section III-C) and from the soft-quantile structure of (5). Using all demand seen so far, scaling each product's profit in the loss and training on more products brought the regret to 0.80%. On 3,000 validation products the final policy reaches 0.78%.

Appendix C retains negative results. A tested larger level network fitted the analytic fractile less accurately; this alone does not establish lower control reward. Imitation warm-starting and a longer training horizon did not improve the reported outcomes. A mismatch between static imitation targets and expanding-window deployment may explain the warm-start result, but that cause was not isolated.

B. Comparison at Matched Information

Table IV and Fig. 5 match the demand information available to each policy. The expanding window gives the lowest regret for all four policies. Gamma fit performs best in every window; the empirical quantile also outperforms the final alternative (0.27% versus 0.80%). Thus the alternative does not improve on the strongest classical rules with matched information.

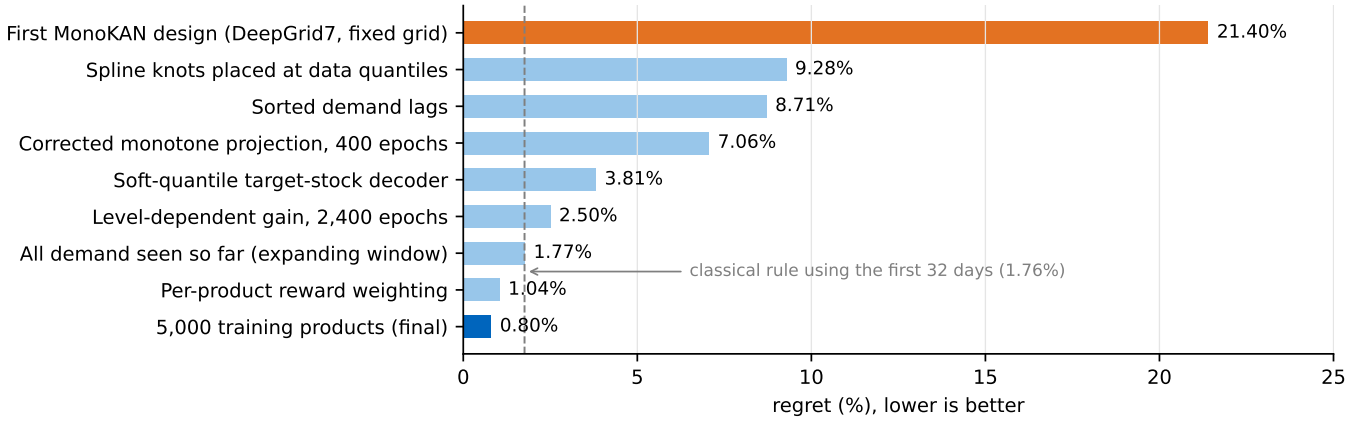


Fig. 4. Regret of the alternative design after each cumulative change, three seeds each. All variants were designed to enforce monotonicity; the projection defect of Section III-C was corrected at the fourth step.

TABLE IV
REGRET OF EVERY POLICY WITHIN EACH KIND OF INFORMATION

Policy	First 32 days, never updated	Last 32 days, updated daily	All demand seen so far
Classical Gamma fit[a]	1.35%	1.25%	0.18%
Classical empirical quantile	1.76%	1.90%	0.27%
Alternative design, KAN without certificate	2.37%	1.96%	0.87%
Alternative design, MonoKAN	2.04%	1.83%	0.80%

Final setup (5,000 training products), three seeds, the same 300 validation products. The last two rows are the same policy with and without the certificate. [a]Also told that demand is Gamma distributed.

Differences in window size must not be credited to the neural architecture.

C. The Same KAN With and Without the Certificate

The last two rows of Table IV are the core test of this design: the same policy, the same 1,892 parameters, the same training and data, once with and once without the certificate. MonoKAN has the lower regret in all three windows. Table V adds paired tests on profit. In the final setup with all demand, the mean profit difference is -0.44 per period (95% CI -1.97 to +1.08, $p = 0.569$): we detected no significant difference, which is not the same as showing that the profits are equal. MonoKAN is ahead on 61% of products (Wilcoxon $p = 0.0001$). With the last 32 days the gain is small but consistent; with the fixed estimate it is borderline, since only the t-test finds it. In a smaller setup matched to the main design (1,000 products, profit objective, last 32 days) the certified model earned more: +5.90 per period (95% CI +3.32 to +8.48), MonoKAN ahead on 72% of products, and regret 2.26% against 3.35%.

The decoder supplies inventory monotonicity in both arms, so these comparisons concern the constrained economic mapping. They do not explain why its profit effect changes with training setup. Earlier controls also show no detected mean difference against a parameter-matched MLP (-0.08, $p = 0.94$), and a gain of +6.73 over KAN with a running-mean decoder ($p = 0.0001$). Those blocks use different settings and should not be pooled with Table V.

D. Interpretability, Speed and Data Needs

With only four inputs the level network is easy to read (Fig. 6(a)). The learned target stock rises with price and penalty and falls with purchase and holding cost, as certified. The learned percentile u also tracks the newsvendor fractile β of (2) closely, with correlation 0.982 (Fig. 6(b)), although the fractile was never given to the model as an input, an initial value or a training target. This shows agreement in the level, not that the policy equals the textbook one: the quantile-based, order-up-to structure was supplied by design, and the decoder has its own learned smoothing and gain.

The final policy has 1,892 parameters, against 15,297 for the earlier DeepGrid7 direct-target model. It is still slower per decision than a feed-forward network of the same size (0.47 ms against 0.06 ms on a CPU), because spline evaluation is more expensive than a matrix product. These measured latencies are small relative to a daily ordering interval. The experiment does not show a general data-efficiency advantage. With 100 training products MonoKAN is worse than a parameter-matched MLP (Fig. 7). Across the three tested training-set sizes (three seeds each), MonoKAN's regret fell steadily, while the MLP's rose again at 1,000 products; other sizes, budgets or initialisations may behave differently.

VII. THE TWO DESIGNS SIDE BY SIDE

Table VI compares the designs at 1,000 training products, 2,400 updates, an unweighted profit objective and three seeds, using the last 32 observations. The remaining protocol differences are described in Section IV-C.

The target-stock design has substantially lower regret in this comparison. Its decoder encodes a known structure, but archi-

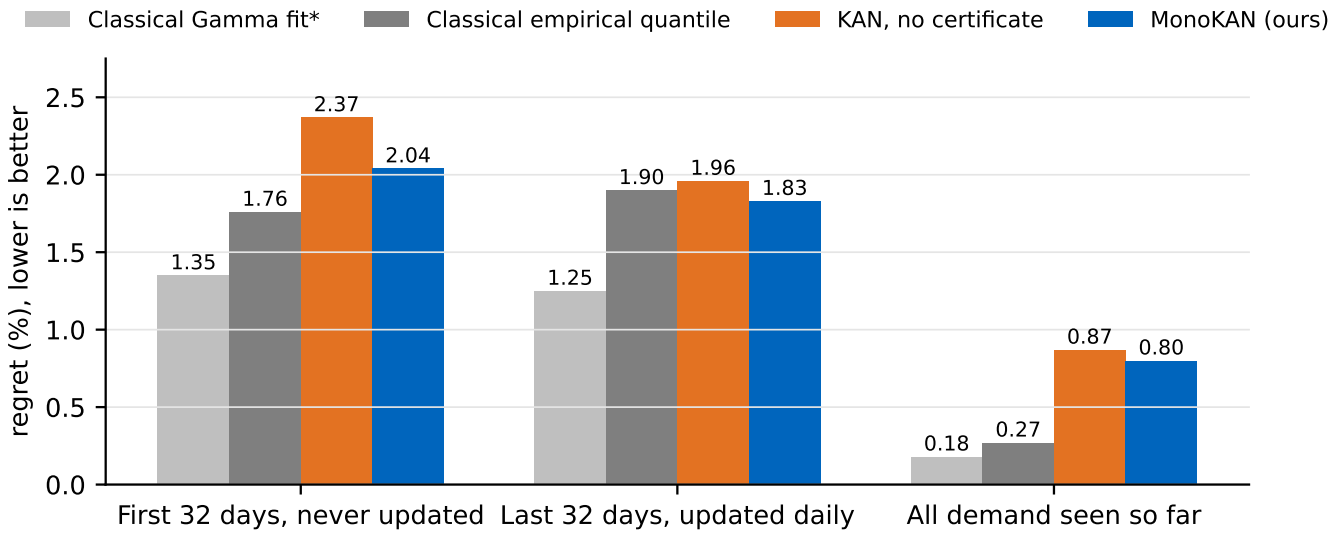


Fig. 5. The data of Table IV. The first window is the paper's benchmark definition, the second is what a policy in daily use sees, and the third is the final alternative policy's information.

TABLE V
MONOKAN MINUS THE SAME KAN WITHOUT THE CERTIFICATE, ALTERNATIVE DESIGN

Setting	Difference	95% CI	t-test p	Wilcoxon p	MonoKAN ahead	Reading
Final setup, all demand seen so far	−0.44	[−1.97, +1.08]	0.569	0.0001	61%	no significant mean difference; ahead on most products
Final setup, last 32 days	+1.81	[+0.16, +3.46]	0.032	$<$ 0.0001	62%	small but consistent
Final setup, first 32 days	+2.64	[+0.17, +5.10]	0.036	0.464	51%	small and borderline
1,000 products, profit objective, last 32 days	+5.90	[+3.32, +8.48]	$<$ 0.0001	$<$ 0.0001	72%	ahead on both tests

Profit per period, paired over the 300 products with rewards averaged over seeds.

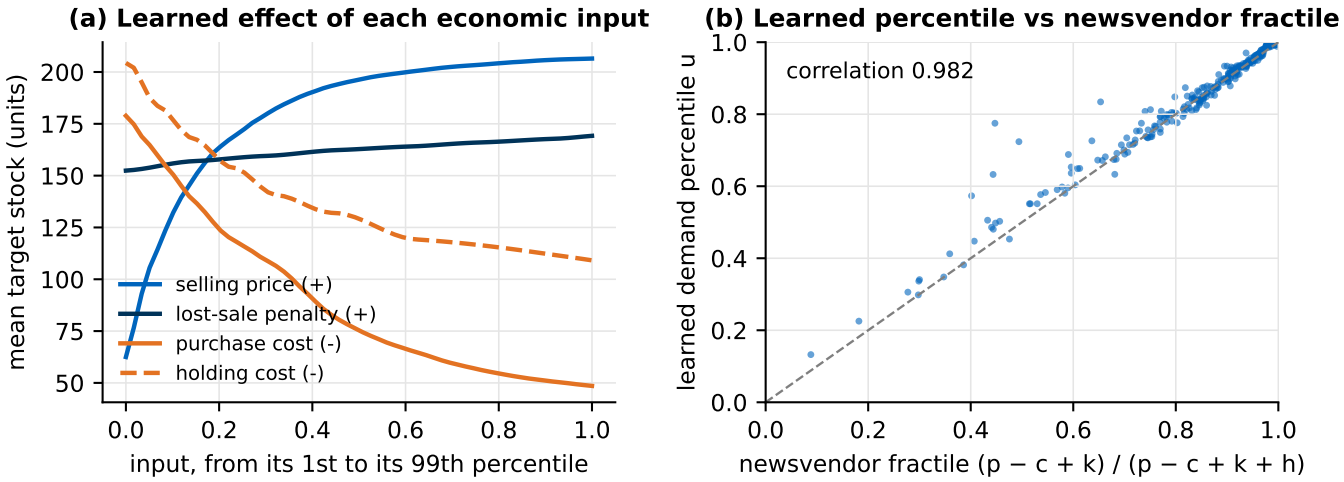


Fig. 6. (a) Mean target stock as each economic input sweeps from its 1st to its 99th percentile, with the other inputs at their observed values; the certified direction is in brackets. (b) Learned percentile against the analytic newsvendor fractile for the 300 validation products. Both panels use the stored seed-2025 checkpoint; mean curves illustrate its responses but are not, by themselves, a global certificate.

tecture, initialization, checkpoint frequency and the evaluation guard also differ. The result therefore supports the combined policy design, not a causal claim that constraining more inputs necessarily costs more profit. With an expanding window, the corresponding 1,000-product alternative reaches 1.62% regret.

The direct-quantity formulation remains the main test of in-

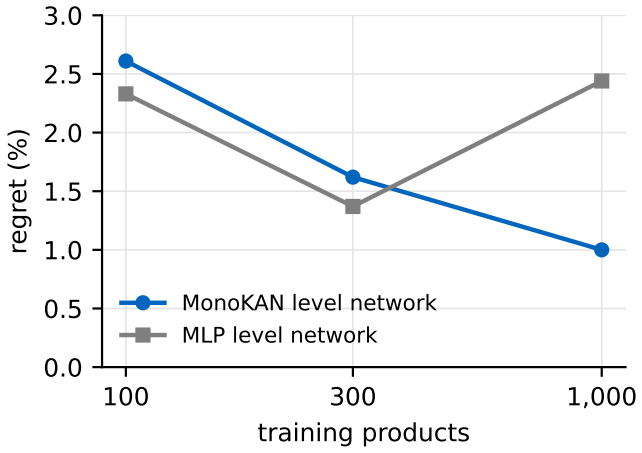


Fig. 7. Regret of the alternative design against the number of training products, MonoKAN and a parameter-matched MLP level network.

TABLE VI
BOTH DESIGNS AT MATCHED TRAINING SIZE, UPDATE BUDGET AND DEMAND WINDOW

	Main design: direct quantity	Alternative: target stock
Who enforces "more stock, never a larger order"	the certificate	the subtraction $(S - I)^+$
Without the certificate: products breaking the stock rule	8.7–51.3% per run	0% (impossible)
Certified minus unconstrained KAN under the same initialization scheme, profit per period	-14.11 ($p = 0.0008$) from the constant start; +548.15 from the open start	+5.90 ($p < 0.0001$)
Regret of MonoKAN	14.60% (constant start), 15.71% (open)	2.26%

ventory monotonicity because it does not obtain that rule from a decoder. The alternative exploits a known solution structure, which can be inappropriate in other settings. For example, Maggiar et al. [1, Table 4] report DRL profit improvements of 0.79%–5.68% over their standard base-stock benchmark with lead times of 2–7 periods. This is their baseline-relative profit improvement, not the regret measure used here. Their warning about tailored structures [1, Sec. 5.3.1] motivates testing broader settings; our experiments do not establish an advantage there.

VIII. DISCUSSION

A. Why Impose Monotonicity if Profit Is the Objective?

Restricting a fixed policy class cannot increase its maximum training objective. Finite-data training and optimization, however, need not find the same solution with and without constraints. The open-start comparison illustrates that a constraint can coincide with substantially better training outcomes; the constant-start comparison shows a modest mean-profit cost. Neither establishes a universal effect.

We treat monotonicity as an explicit behavioural requirement. It excludes selected reversals beyond the finite states checked after training, while profit and regret measure operational performance on the evaluation products. These are different criteria. The current experiments support direction guarantees, not general claims of superior profit, sample efficiency or robustness.

B. Limitations

- *A simple setting.* Demand is stationary, orders arrive immediately and products are synthetic. This is exactly the setting in which order-up-to is optimal, and it favours the alternative design. We have not tested changing demand, lead times or real data.
- *Scale.* We train on 1,000 to 5,000 products, compared with 40,000 in the paper, and use three seeds per experiment. Our absolute profit levels are therefore not comparable with the paper's.
- *Selection on the validation products.* Checkpoints are selected, and the method was developed, on the same 300 validation products that we report on. Pairing on shared products and demand paths reduces product-level noise, but it does not remove this selection bias. The final alternative policy keeps its standing on 3,000 products (0.78%), but that set contains the original 300 and is not a fully independent test (Appendix C).
- *Two starts, one evaluation cap.* The main design's comparison depends on the initialisation, and we tested two starts; other starts, learning rates or budgets could move the cost of the certificate either way. Its evaluation caps orders, which its training does not (Section IV-A); the cap applies to all arms alike and does not change our conclusions (Section V-B).
- *Response shape is unexplained.* Average fitted stock reactions are weaker than the reference, but local slopes can also be too steep. Limited training-state coverage is one possible explanation; it has not been isolated experimentally.
- *Model family.* The stock-rule breaks are a property of the KAN family in our runs. The paper's FCNN did not break the rule, so how often an unconstrained model breaks it depends on the model.
- *Training setup.* The main design has not been tested with the alternative's larger training set and scaled objective. Whether these changes help direct-quantity policies remains unknown.

IX. CONCLUSION

The experiments distinguish a monotone direct-order network from a policy whose decoder already encodes the inventory response. In the direct design, MonoKAN prevents wrong-direction responses, but the reward comparison with unconstrained KAN depends strongly on initialisation. The guarantee controls direction, not reaction magnitude: the reported average fitted slopes do not establish a pointwise slope bound. In the structured alternative, inventory monotonicity comes

TABLE VII
COMMANDS THAT REPRODUCE THE RESULTS

Result	Command

Main design, all five arms (Tables II, III, VI; Figs. 2, 3)	`python scripts/run_direct_quantity_comparison.py train --out outputs/direct_quantity`
	`python scripts/run_direct_quantity_comparison.py summarize outputs/direct_quantity`
	`python scripts/run_direct_quantity_comparison.py interpret outputs/direct_quantity`
	`python scripts/run_direct_quantity_comparison.py curves outputs/direct_quantity`
	`python scripts/run_direct_quantity_comparison.py cap outputs/direct_quantity`

Final alternative policy	`python scripts/run_soft_quantile_monokan.py --seed 2025 --output-dir outputs/ts_monokan/seed_2025`
Same policy without the certificate	`... --set monokan.level_network=kan`
Last 32 days only / first 32 days only	`... --set monokan.window_mode=sliding` / `... --set monokan.roll_history=false`
Setup matched to the main design (Table VI)	`--config configs/paper_lost_sales_soft_quantile_monokan_profit.yaml --set epochs=2400 --set idp_benchmark.compact.reward_epochs=2400 --set idp_benchmark.compact.validation_interval=150 --set monokan.window_mode=sliding`
Classical rules in every window (Table IV)	`python scripts/run_classical_windows.py`
Regret and paired tests (Tables IV, V)	`python scripts/compare_target_stock_runs.py summary outputs/ts_monokan outputs/ts_kan` `python scripts/compare_target_stock_runs.py pair outputs/ts_monokan outputs/ts_kan`
Interpretability (Fig. 6)	`python scripts/run_monokan_interpretability.py ...`
This edited report, with its accompanying `figures/` directory	`pdflatex main.tex` `pdflatex main.tex`

TABLE VIII
TRAINING AND EVALUATION SETTINGS

Setting	Main design	Alternative, final	Alternative, matched
Configuration (`configs/paper_lost_sales_*.yaml`)	`rolling_direct_target_dev_1k_normalized`	`soft_quantile_monokan`	`soft_quantile_monokan_profit`
Training products	1,000	5,000	1,000
Optimizer updates (logged as epochs)	2,400	2,400	2,400
Batch size	512	512	512
Checkpoint interval (updates)	300	150	150
Objective	total profit	mean scaled profit, $R_i/(p_i\hat{\mu}_i T)$	total profit
Demand information	last 32 days (order scaled by the initial 32-day mean)	all demand seen so far	last 32 days
Network	KAN 37–8–1, 5 intervals	level network 4–8–8–1, 7 intervals	
Output initialisation	constant or open, both reported (Section IV-A)	same level-calibration scheme for MonoKAN and KAN	
Spline knots	placed at quantiles of the training inputs		
Optimiser	Adam, learning rate 0.001, best validation checkpoint		
Training horizon	100 periods, fixed demand paths, terminal stock credited at purchase cost		
Evaluation	300 validation products, 520 periods, first 20 discarded, identical demand paths for all policies		
Order cap	in evaluation only: stock after ordering at most $4\times$ the initial 32-day mean	none	
Seeds	2025, 2026, 2027		

from the order-up-to decoder, while MonoKAN constrains the economic mapping. This alternative reached 0.80% regret, with no detected mean profit difference from its matched unconstrained KAN. Classical rules using the same expanding information remained stronger in this setting. Differences between the two designs cannot be attributed solely to the number of constrained inputs.

Further experiments should isolate the inventory constraint while keeping the other constraints and training protocol fixed, and evaluate whether wider inventory-state coverage or explicit slope restrictions improve policy behaviour. A bound between -1 and 0 would limit steep responses but would not itself force a stronger response to stock. These questions, and evaluation on an independent protected population, remain open; changing demand or lead times would require a separate study of whether the imposed policy structure remains appropriate.

APPENDIX A
REPRODUCING THE RESULTS

Commands in Table VII run from the repository root. Install the project with `python -m pip install -e ".[dev]"` and run tests with `python -m pytest -q`. Target-stock training commands run one seed at a time and are repeated for seeds 2025, 2026 and 2027. Training is not required to inspect the saved results. The twelve direct-design KAN summaries are in `report/data/direct_quantity`; their checkpoints are in `outputs/direct_quantity_matched_init`. The three final alternative-policy runs are in `outputs/soft_quantile_monokan` and can be summarised with `python scripts/compare_target_stock_runs.py summary outputs/soft_quantile_monokan`. The submitted archive does not include the original report generator. This edited report is a standalone `main.tex` with six accompanying figure PDFs. Some earlier FCNN and information-window per-run outputs were not retained; their tabulated results therefore cannot all be rechecked from the submitted archive alone.

The generic saved target-stock diagnostic for the final soft-quantile wrapper includes an extra historical-mean factor and should not be interpreted as its target level. Its empty spline-certification CSV is also not evidence of certification. The structural argument instead requires the signed network paths, spline constraints and monotone decoder described in Section III; empirical plots alone do not establish a global guarantee.

APPENDIX B
PROTOCOL DETAILS

Table VIII lists the training and evaluation settings of both designs.

APPENDIX C
ADDITIONAL RESULTS

Table IX evaluates the final alternative policy on a 3,000-product superset, Table X gives its profit and regret by the number of training products, and Table XI lists the changes that did not improve it.

TABLE IX
ADDITIONAL EVALUATION ON 3,000 VALIDATION PRODUCTS

Policy	Profit / period	Regret
Omniscient base stock	4,693.34	0.00%
Soft-quantile MonoKAN, 5,000 products	4,679.59	0.78%
Soft-quantile MonoKAN, 1,000 products	4,678.87	1.28%
Classical Gamma fit, first 32 days	4,671.74	1.46%
Classical empirical quantile, first 32 days	4,651.87	2.34%

The 3,000 products are a superset of the 300 used for checkpoint selection. The absolute profit level differs from the main tables because the product draw differs; the ordering of policies is unchanged.

TABLE X
ALTERNATIVE DESIGN: PROFIT AND REGRET BY NUMBER OF TRAINING PRODUCTS

	MonoKAN level network		MLP level network	
Products	Profit / period	Regret	Profit / period	Regret
100	4,279.35	2.61%	4,285.05	2.33%
300	4,282.60	1.62%	4,284.38	1.37%
1,000	4,284.08	1.00%	4,278.53	2.44%

Three seeds, 2,400 epochs each.

TABLE XI
CHANGES THAT DID NOT IMPROVE THE ALTERNATIVE DESIGN

Change	Outcome
Imitation warm start from the classical rule	8.61% regret after 200 epochs. Static imitation and expanding-window deployment differ; whether that mismatch causes the degradation was not isolated.
Larger level network (3×16, 12 intervals)	Worse fit of the newsvendor fractile (RMSE 0.033 against 0.026 for 2×8).
Training horizon 250 instead of 100	No gain (2.06% regret).
Running mean on top of the expanding window	Worse by 6.8 profit per period; additional smoothing is a possible explanation.
Per-product normalisation of prices	Rejected: it breaks the certificate, because raising the price moves the cost ratios in conflicting directions.

REFERENCES

[1] A. Maggiar *et al.*, "Structure-informed deep reinforcement learning for inventory management," 2025, arXiv:2507.22040. [Online]. Available: https://arxiv.org/abs/2507.22040

[2] J. Sill, "Monotonic networks," in *Advances in Neural Information Processing Systems 10*. Cambridge, MA, USA: MIT Press, 1998, pp. 661–667.

[3] S. You, D. Ding, K. Canini, J. Pfeifer, and M. Gupta, "Deep lattice networks and partial monotonic functions," in *Advances in Neural Information Processing Systems 30*, 2017.

[4] A. Polo-Molina, D. Alfaya, and J. Portela, "MonoKAN: Certified monotonic Kolmogorov–Arnold network," *Neural Networks*, vol. 196, Art. no. 108278, 2026, doi: 10.1016/j.neunet.2025.108278.

[5] Z. Liu *et al.*, "KAN: Kolmogorov–Arnold networks," 2024, arXiv:2404.19756. [Online]. Available: https://arxiv.org/abs/2404.19756

[6] P. H. Zipkin, *Foundations of Inventory Management*. McGraw-Hill, 2000.

[7] F. N. Fritsch and R. E. Carlson, "Monotone piecewise cubic interpolation," *SIAM J. Numer. Anal.*, vol. 17, no. 2, pp. 238–246, 1980.